

EDUGENIE AI

AI-Powered Learning Assistant

PROJECT DOCUMENTATION

Team Members

1.Cenobia Samreen -TL

2.Beula

3.Brinda Sri

4.Dhanalakshmi

EduGenie AI helps learners ask questions and receive AI-generated educational answers in a simple web interface.

1. Project Structure

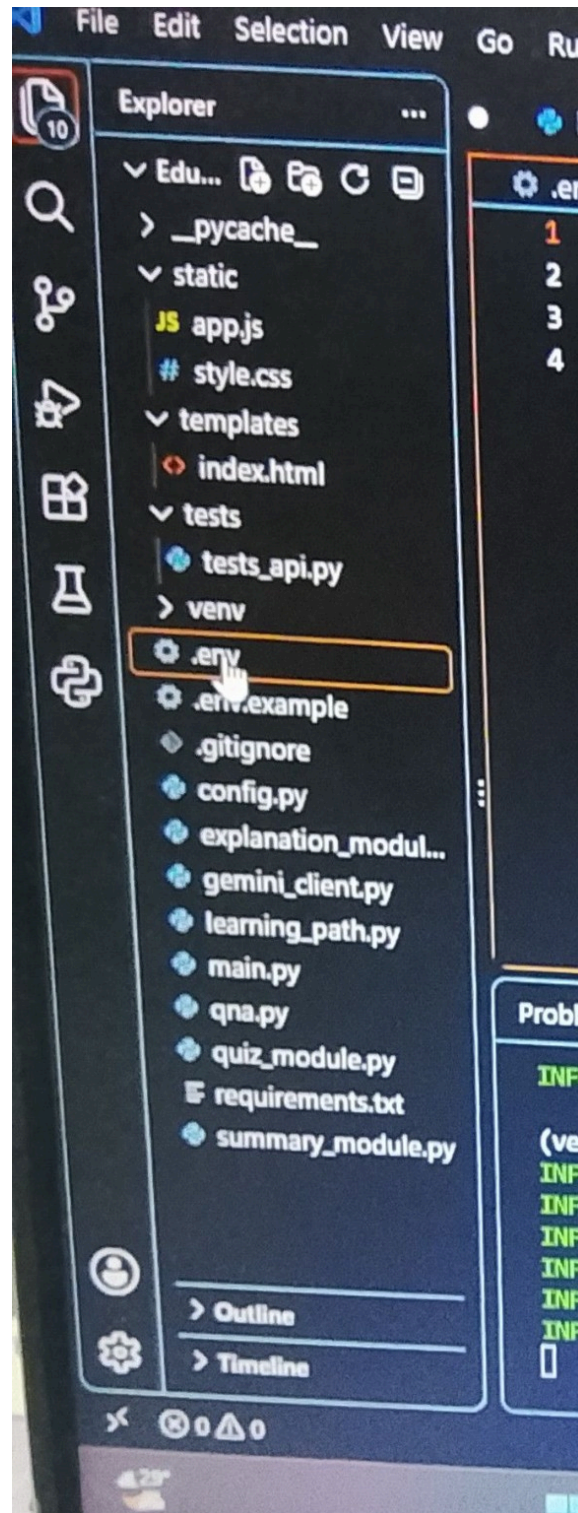


Figure 1: EduGenie AI project structure in Visual Studio Code

The project is organized into frontend files, Python backend modules, testing files, configuration files, and a virtual environment.

- static/ contains the frontend JavaScript and CSS files used to control the appearance and interaction of the website.

- `templates/` contains `index.html`, which provides the main webpage interface.
- `tests/` contains `tests_api.py` for checking important API routes and validation behavior.
- `main.py` connects the web interface with the backend and provides the application routes.
- `quiz_module.py`, `qna.py`, `summary_module.py`, `explanation_module.py`, and `learning_path.py` provide separate learning-related AI features.
- `requirements.txt` lists the Python packages required to run the project.
- `.env` stores the Gemini API key and other private configuration values; it should not be uploaded publicly.

2. style.css – Website Styling

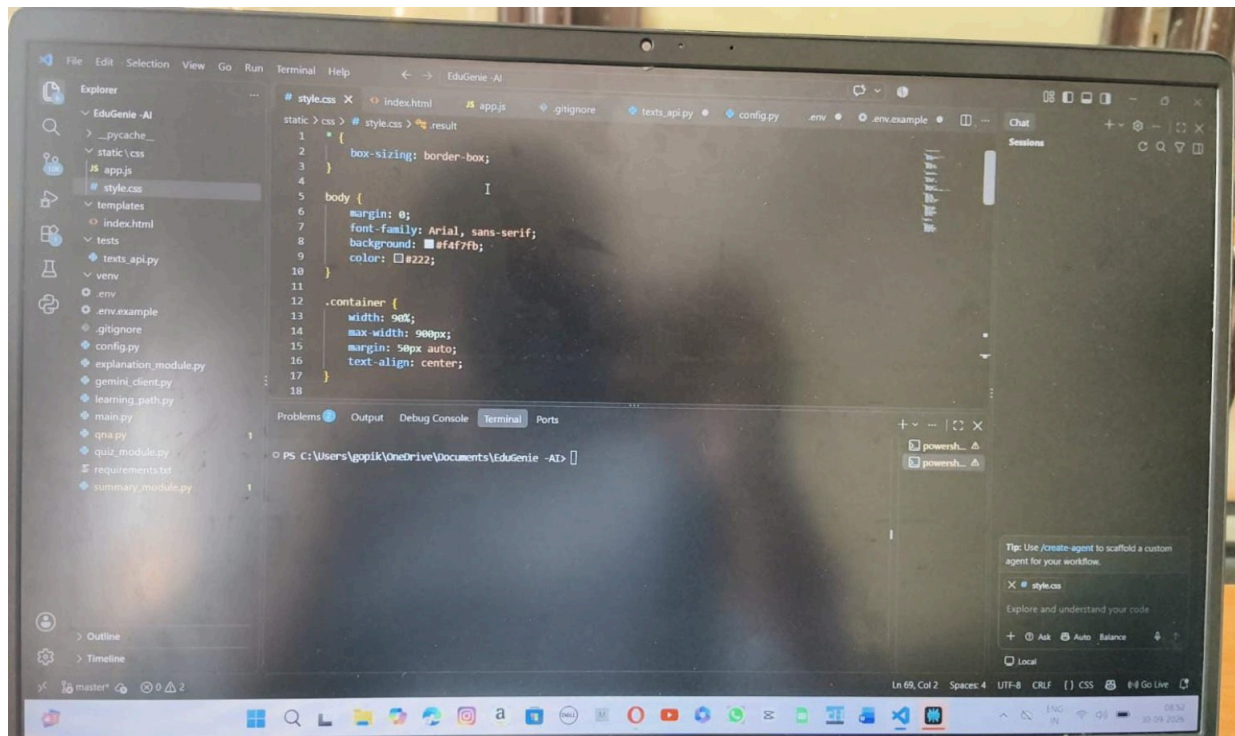


Figure 2: CSS code used to style the EduGenie AI webpage

The style.css file controls the visual appearance of the EduGenie AI website.

- The box-sizing rule makes element sizing easier to manage.
- The body section removes the default margin, sets Arial as the font, and applies a light background and dark text.
- The .container class controls the page width, maximum width, automatic margins, and centered alignment.
- These styles provide a clean and simple layout for the learning assistant interface.

3. index.html – Main Webpage

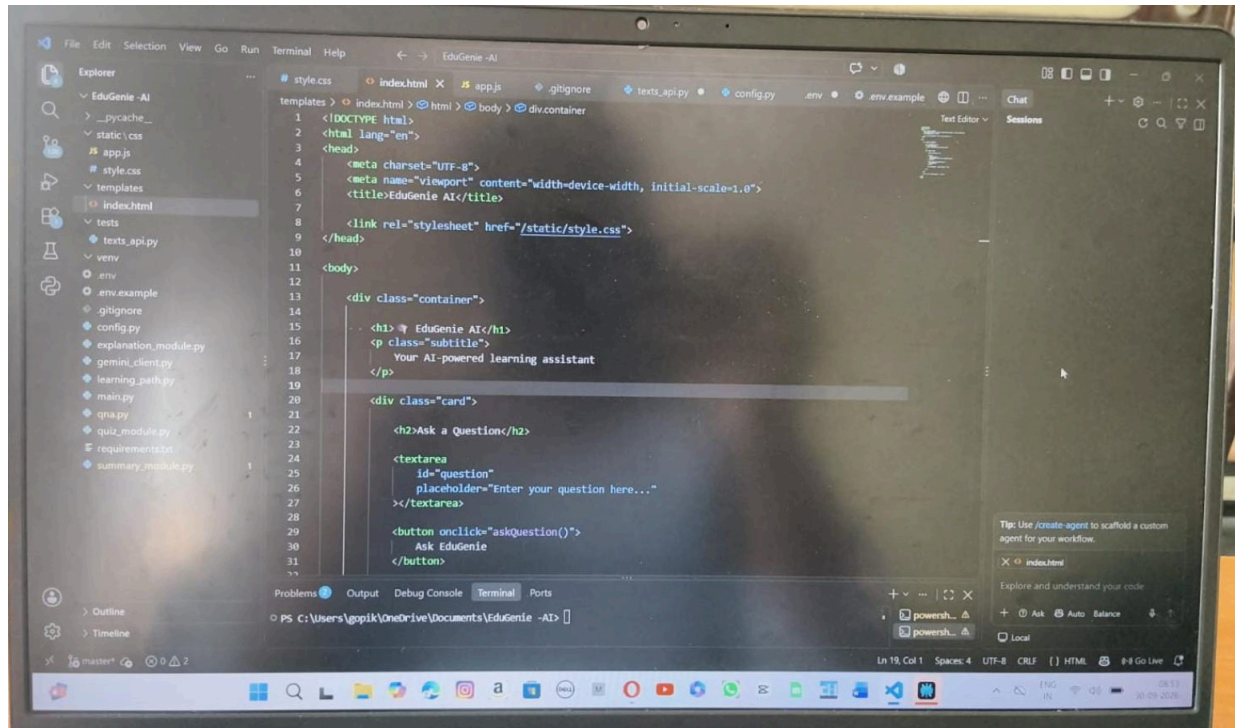


Figure 3: HTML code for the EduGenie AI user interface

The index.html file creates the main webpage that the learner interacts with.

- The page defines the document type, language, character encoding, and responsive viewport.
- It links the webpage to /static/style.css so the CSS styling is applied.
- The page displays the EduGenie AI title and the subtitle 'Your AI-powered learning assistant'.
- A question text area allows the learner to type a question.
- The 'Ask EduGenie' button calls the askQuestion() JavaScript function to send the question for processing.

4. tests_api.py – API Testing

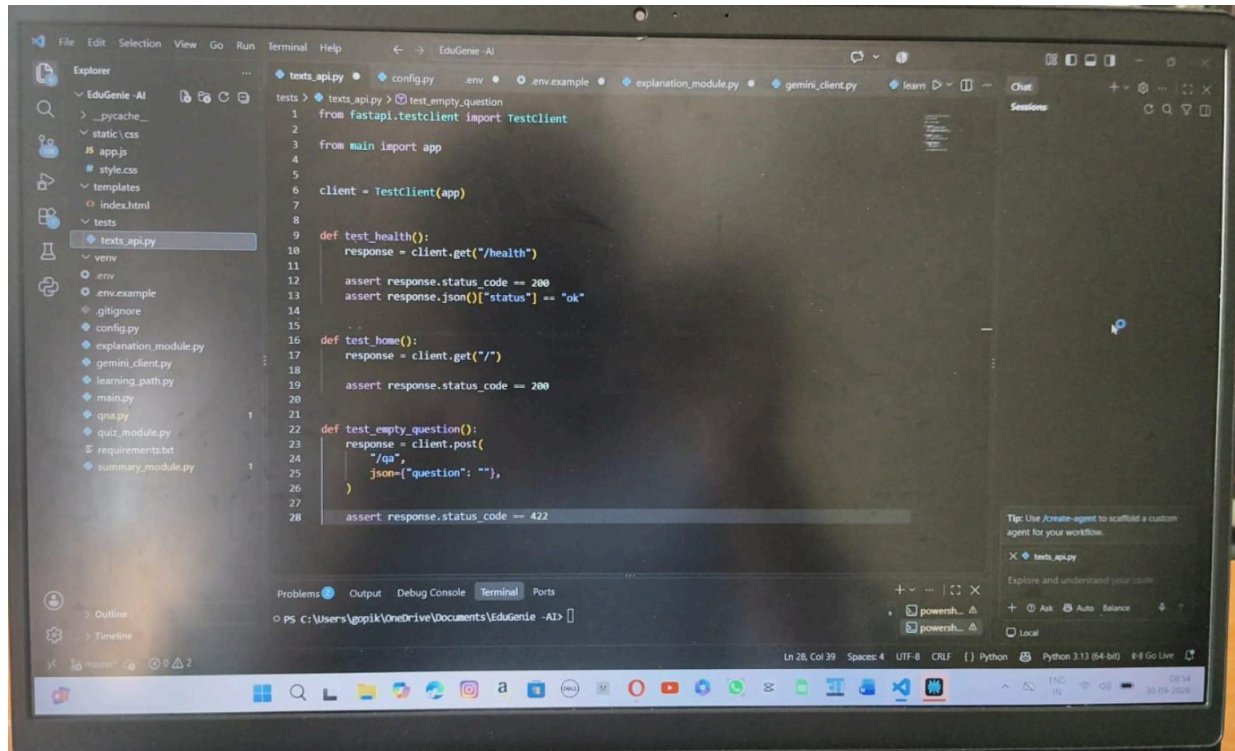
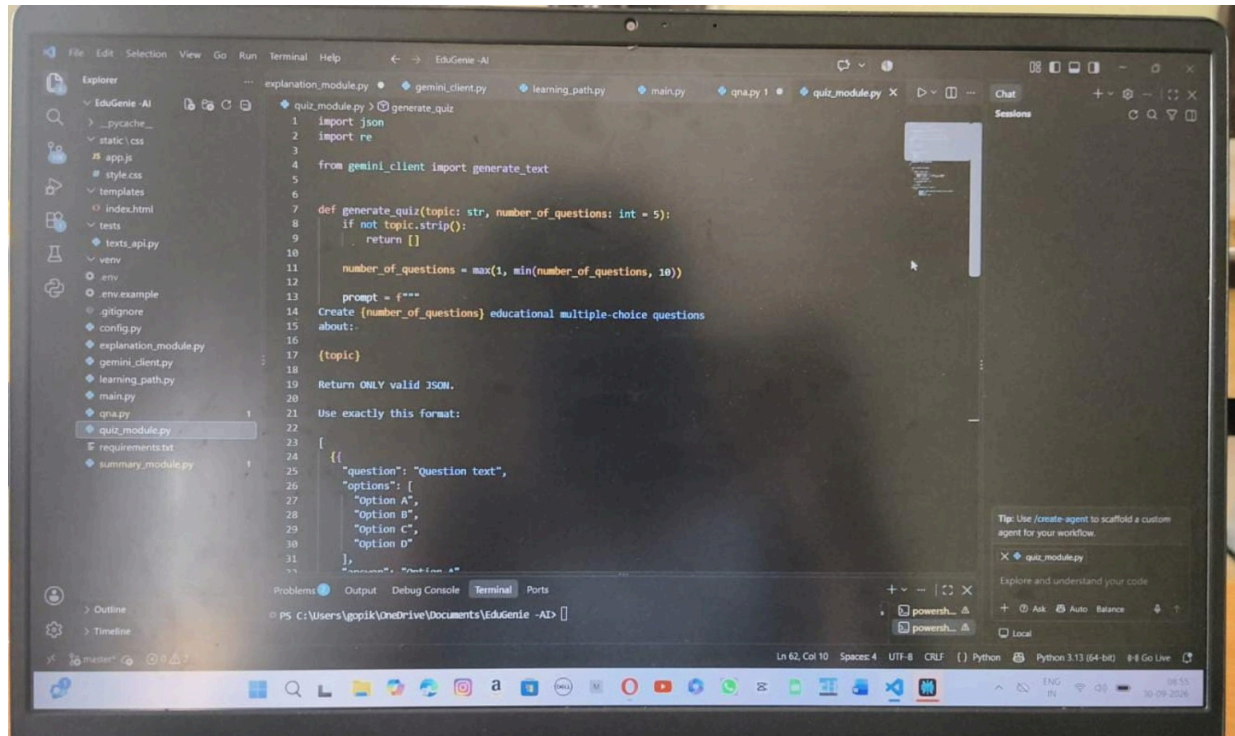


Figure 4: API test cases for the EduGenie AI application

The tests_api.py file checks whether important parts of the FastAPI application are working correctly.

- TestClient is used to send requests to the FastAPI application without opening the website manually.
- test_health() checks the /health endpoint and verifies that the returned status is 'ok'.
- test_home() checks that the main / route responds successfully.
- test_empty_question() sends an empty question to the Q&A endpoint and verifies that the application rejects invalid input with HTTP 422.
- These tests help detect basic API and input-validation problems before deployment.

5. quiz_module.py – AI Quiz Generation



```
1 import json
2 import re
3
4 from gemini_client import generate_text
5
6
7 def generate_quiz(topic: str, number_of_questions: int = 5):
8     if not topic.strip():
9         return []
10
11     number_of_questions = max(1, min(number_of_questions, 10))
12
13     prompt = f"""
14 Create {number_of_questions} educational multiple-choice questions
15 about:
16 {topic}
17
18 Return ONLY valid JSON.
19
20 Use exactly this format:
21 [
22   {
23     "question": "Question text",
24     "options": [
25       "Option A",
26       "Option B",
27       "Option C",
28       "Option D"
29     ],
30     "correct": "Option A"
31   },
32   ...
33 ]"""
```

Figure 5: Code for generating educational multiple-choice questions

The quiz module uses the Gemini-powered text generation function to create quizzes from a given topic.

- generate_quiz() accepts a topic and the required number of questions.
- The number of questions is restricted to a safe range, from 1 to 10.
- A structured prompt asks the AI to create educational multiple-choice questions.
- The prompt requests valid JSON so the generated result can be processed by the application.
- Each question is designed with answer options and a correct answer, making the module useful for self-assessment.

6. requirements.txt – Required Python Packages

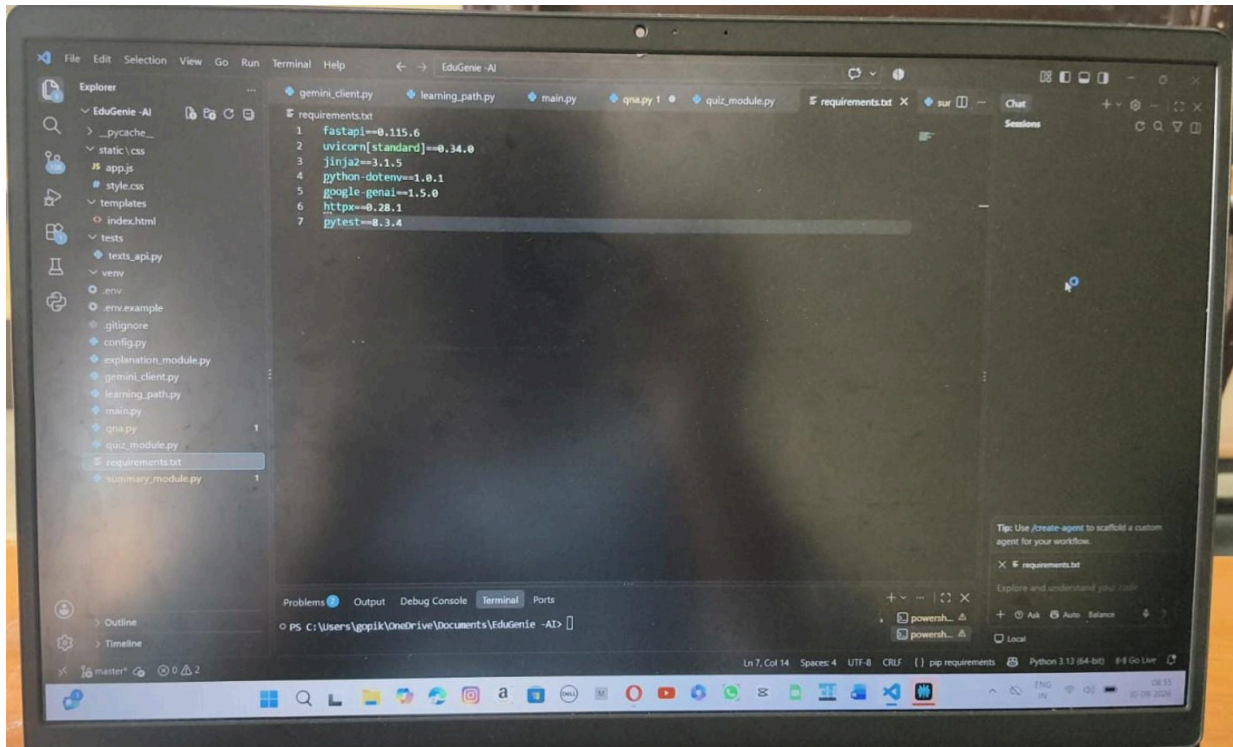


Figure 6: Python dependencies listed in requirements.txt

The requirements.txt file records the libraries needed to install and run the EduGenie AI backend.

- FastAPI is used to build the web API.
- Uvicorn runs the FastAPI application server.
- Jinja2 supports HTML template rendering.
- python-dotenv loads configuration values from the .env file.
- google-generativeai provides access to Google's Gemini generative AI services.
- httpx supports HTTP communication and pytest is used for automated testing.
- Installing the packages from requirements.txt helps keep the project environment consistent.

7. summary_module.py – AI Text Summarization

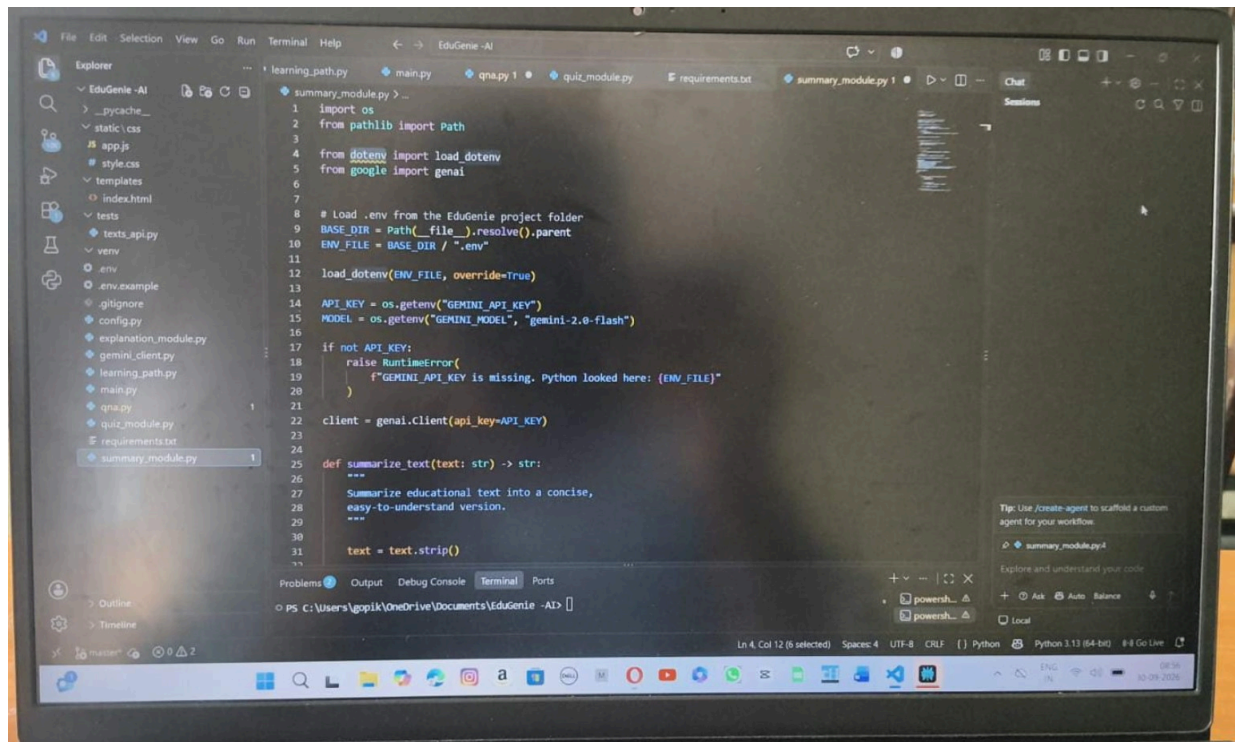


Figure 7: Code for the EduGenie AI summary module

The summary module converts longer educational text into a concise, easy-to-understand summary using Gemini AI.

- The module loads environment variables from the EduGenie project .env file.
- It reads the GEMINI_API_KEY and GEMINI_MODEL values needed to communicate with the AI service.
- If the API key is missing, the program raises a clear error instead of continuing without authentication.
- The summarize_text() function accepts educational text and prepares it for concise summarization.
- This feature helps students understand lengthy study material quickly.

8. EduGenie AI Website – How to Generate an Answer

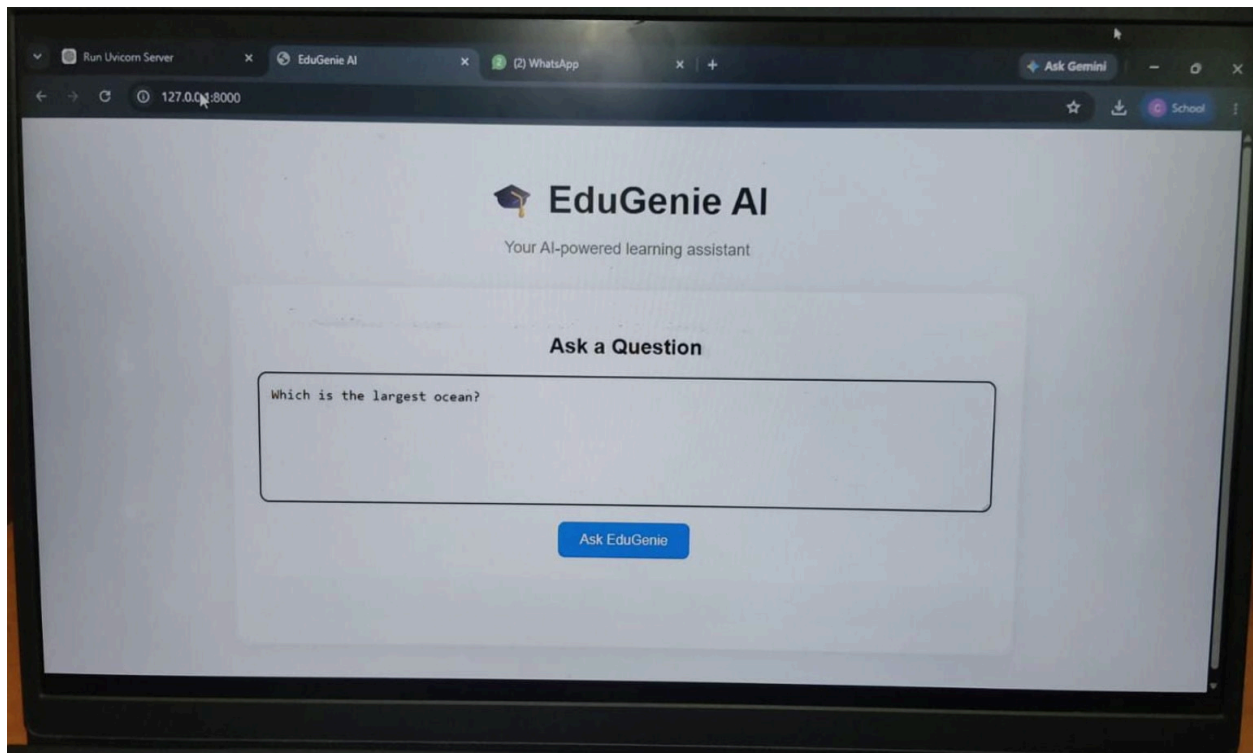


Figure 8: EduGenie AI website running locally

The final website provides a simple interface through which learners can ask educational questions and receive AI-generated answers.

- Step 1: Open the EduGenie AI website in the browser while the FastAPI/Uvicorn server is running.
- Step 2: In the 'Ask a Question' text box, type the question you want to learn about.
- Step 3: Click the 'Ask EduGenie' button.
- Step 4: The question is sent to the backend, where the AI module processes the request.
- Step 5: The Gemini-powered system generates an educational response and returns it to the website.
- Step 6: Read the generated answer and use it as a learning aid.

Example question shown: “Which is the largest ocean?”